A Two-Layer Emergency Escalation Gate for Tool-Augmented LLM Agents: Design and Implementation of a Terminal-Based Patient Intake System

Kaustubha Venkata Eluri
Northeastern University

***Abstract*—Large language models (LLMs) are increasingly proposed as conversational front ends for safety-adjacent domains such as healthcare scheduling and intake, yet LLMs alone are unreliable sources of factual data and cannot be trusted to gate safety-critical decisions on their own. This paper documents the design of a terminal-based patient intake and scheduling agent built on GPT-4o that separates conversational reasoning from data-bearing operations: the LLM drives the dialogue, while deterministic Python tools own address validation (via the Google Maps Geocoding API) and provider ranking. The system's central safety mechanism is a two-layer emergency escalation gate, in which a system-prompt-level instruction to redirect emergency symptoms to 911 is reinforced by a hard refusal at the tool layer, so that even a failure of the LLM's classification does not result in scheduling being offered for an emergency case. The system also ships a keyless demo mode that exercises the identical tool layer as the production path, guaranteeing that the demonstrated behavior cannot diverge from the deployed behavior. We describe the architecture, the escalation gate, the demo-production parity design, and we candidly discuss the project's limitations, since it is an engineering demonstration rather than a clinically validated system. We argue that the honest documentation of these limitations is itself evidence of engineering maturity, and we outline what a rigorous clinical evaluation of such a system would require.**

***Index Terms*—LLM agents, tool-augmented language models, function calling, safety guardrails, healthcare scheduling, human-AI interaction**

I. Introduction

Conversational large language models are fluent, flexible, and increasingly used to front end structured workflows — booking appointments, answering support queries, triaging requests. This flexibility is attractive precisely in domains, such as patient intake, where the input is unstructured natural language (symptoms, addresses, preferences) but the downstream action (scheduling a slot, validating an address, deciding whether to escalate) must be correct and auditable.

The risk is that an LLM used naively as an end-to-end decision maker will hallucinate: it may invent an address that does not exist, fabricate a provider or an available slot, or make an inconsistent judgment about the urgency of a set of symptoms on different runs of an otherwise-identical conversation. In a domain like healthcare scheduling, an ungrounded output is not merely an inconvenience — a missed or mishandled emergency-symptom case has direct safety consequences.

This paper documents a small, single-file, terminal-based patient intake agent that treats this risk as a first-class design constraint. The system's guiding principle, stated plainly in its own documentation, is that *the LLM drives the conversation, but deterministic tools own every data-bearing operation*. Address validation is delegated to the Google Maps Geocoding API; provider ranking is delegated to a rule-based, in-memory ordering function; and — most importantly for safety — emergency escalation is enforced not by trusting the LLM's judgment alone, but by a second, independent gate implemented in the tool layer itself.

The contribution of this paper is not a novel model or a large-scale empirical evaluation. It is a documented, working instance of an architectural pattern for tool-augmented LLM agents in a safety-adjacent domain, together with an explicit two-layer escalation-gate design that we believe generalizes beyond this specific application. We also document, without embellishment, the system's known limitations, since the project's own documentation makes an explicit case that it is a workflow demonstration and not a clinically validated or HIPAA-compliant product.

II. Related Work

The broader idea explored here — augmenting a language model with external tools so that it can retrieve or act on facts it cannot reliably generate itself — is now a common pattern in LLM application design, generally referred to as tool use or function calling. Rather than asking a model to produce a fact (an address, a database record, a computed value) directly in natural language, the model is instead asked to emit a structured call to an external function; the function's real, deterministic result is then fed back into the model's context so that the model's final natural-language response is grounded in that result rather than in the model's own (possibly hallucinated) recollection. Commercial LLM APIs, including OpenAI's chat-completions API used in this project, expose this capability natively as "tool calling," allowing a model to select from a set of declared function schemas during a conversation.

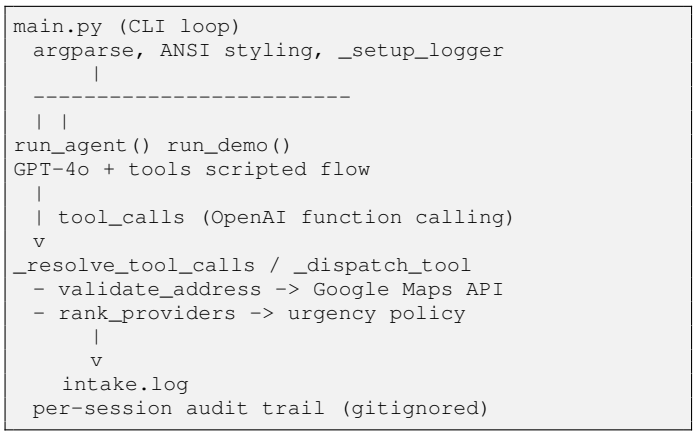

Fig. 1. Layer diagram of the intake agent. The CLI dispatches to either the LLM-driven agent path or the scripted demo path; both paths route data-bearing operations through the same tool-dispatch layer, and both are logged to a common audit trail.

A second, related line of work concerns safety guardrails for LLM-driven agents: techniques for constraining what an LLM-based system is permitted to do, independent of whether the underlying model itself behaves correctly on a given turn. Common patterns include system-prompt-level behavioral instructions, output filtering or classification layers, and hard constraints implemented outside the model — for example, refusing to execute a downstream action unless certain preconditions are verified in code rather than left to the model's discretion. The design described in this paper is best understood as a concrete, minimal instance of this second category: a safety property (never offer to schedule an appointment for an emergency-classified patient) is enforced redundantly at two independent points in the pipeline, one inside the model's instructions and one outside the model entirely, in ordinary Python control flow.

We do not claim novelty in either of these general ideas; both are established practice in contemporary LLM application engineering. The contribution of this paper is the concrete application and documentation of these patterns, together with an explicit two-layer emergency gate, in an end-to-end intake and scheduling system.

III. System Architecture

A. Overview

The system is implemented as a single Python file (`main.py`, approximately 24 KB, roughly twelve top-level functions) with two run modes selected by a command-line flag: a production mode that calls GPT-4o and the Google Maps Geocoding API, and a keyless demo mode (`--demo`) that scripts the same conversational flow without any network calls to an LLM provider. Figure 1 (reproduced from the project's own architecture documentation) shows the layering.

B. The core design principle: the LLM drives, tools own the data

The system prompt walks the patient through a fixed sequence of conversational goals: collecting name and address, eliciting symptoms and an urgency classification (emergency, urgent, or routine), ranking candidate providers according to that urgency, offering a slot, and producing a confirmation summary that is appended to the audit log. Within this sequence, the model is explicitly never permitted to originate the two categories of information that matter for correctness and safety: whether an address is real, and which providers are available and in what order.

Instead, whenever the conversation reaches a point requiring one of these facts, the model emits a structured tool call, which the dispatch layer (`_resolve_tool_calls` / `_dispatch_tool`) intercepts and routes to one of two deterministic Python functions:

- **`validate_address`** (implemented as `_validate_address`) issues an HTTP request to the Google Maps Geocoding API and returns a structured result of the form {`valid: bool, formatted_address: str`}. This both rejects addresses that do not resolve and normalizes the address string used downstream in scheduling and logging.
- **`rank_providers`** (implemented as `_rank_providers`) orders an in-memory table of three mock providers (Primary Care, Family Medicine, and Internal Medicine practitioners) according to a fixed urgency policy: urgent cases place the Internal Medicine provider first with the earliest slot auto-selected; routine cases place Primary Care / Family Medicine first and let the patient choose a slot; emergency cases receive an *empty* provider list, discussed further in Section IV.

The result of each tool call is appended back into the chat message history, and the model's subsequent natural-language turn is generated conditioned on that real result rather than on an unconstrained guess. The project's own architecture decision record (ADR-001) states the underlying rationale directly: the LLM is good at language and bad at facts, so asking GPT-4o to "validate" an address or "rank" providers in free text would produce confidently wrong or non-reproducible outputs, whereas routing those decisions through Python tool calls makes the system auditable and repeatable and prevents the model from inventing a provider that does not exist. Every provider currently in scope is a mock in-memory record; the documented production migration path (ADR-007) is to replace the in-memory table with calls into a real clinic scheduling API, while leaving the tool schema — and therefore the LLM-facing contract — unchanged.

Every user input, agent message, tool call, and tool result is appended, with a timestamp, to a per-session audit log file (`intake.log`), which is excluded from version control (ADR-005) because session contents may contain patient-identifying information such as name, address, and symptoms.

IV. The Two-Layer Emergency Escalation Gate

The system's central safety-engineering contribution is its treatment of emergency classification as a hard, redundantly enforced gate rather than a single point of trust in the model's judgment.

A. Triage classes

The system prompt defines three urgency classes, distinguished by keyword and phrase patterns present in the patient's described symptoms:

- **Emergency** — e.g., chest pain, inability to breathe, signs of stroke, loss of consciousness, severe bleeding, suicidal ideation. Outcome: an immediate directive to call 911; the booking flow terminates entirely; the session is logged with status `ended (emergency)`.
- **Urgent** — e.g., high fever, persistent vomiting, signs of infection, severe pain. Outcome: the Internal Medicine provider is placed first in the ranking, and the earliest available slot is auto-selected.
- **Routine** — e.g., annual check-up, mild cold, prescription refill, general wellness. Outcome: Primary Care / Family Medicine providers are placed first, and the patient selects their own slot.

B. Layer one: the system-prompt instruction

The first layer is an instruction embedded in the system prompt directing the model, upon recognizing an emergency-consistent symptom description, to immediately tell the patient to call 911, decline to proceed with scheduling, and end the session. This layer operates entirely within the LLM's own reasoning: it depends on the model correctly classifying the input and correctly following its instructions on a given turn.

C. Layer two: the tool-layer hard refusal

The second, independent layer lives outside the model, in ordinary Python. The `rank_providers` tool — the only function capable of producing a provider list that could subsequently be booked — is implemented so that it returns an *empty list* whenever it is called (or would be called) with urgency classified as emergency. Concretely, this means that even if the system-prompt-level instruction failed on a given turn and the model attempted to proceed to scheduling for an emergency case, the function the model depends on to obtain a schedulable provider would return nothing to schedule against. The escalation path is therefore not merely a recommendation to the model; it is backed by a control-flow property of the code that the model cannot talk its way around.

The project's own ADR-002 states the rationale explicitly: triage decisions affect human safety, so the escalation path must be unbypassable, and two independent layers are used because either one alone could fail. This pattern — enforcing a safety property once inside model instructions and once again outside the model, in code that does not depend on the model behaving correctly — is, we argue, a useful minimal template for safety-relevant LLM agent design more generally: wherever a single point of failure would be a model's own judgment on a given turn, a second, model-independent check on the same property should be added at the point where an unsafe action would otherwise be executed.

The project's own quality gates make this property directly testable: calling `_rank_providers(urgency="emergency")` and confirming it returns an empty list is listed as one of the system's verification steps, independent of any LLM call.

V. Demo-Production Parity

A recurring risk in demonstrations of LLM-driven systems is that the demo shown to a reviewer behaves differently from — and more favorably than — the system that would actually run in production, because the demo substitutes hand-written or cherry-picked outputs for the real, occasionally messy behavior of the underlying model or its tools. This project addresses that risk directly through its keyless `--demo` mode.

In demo mode, the conversational text is hand-scripted rather than generated by GPT-4o (avoiding the need for an OpenAI API key), and the flow is fixed to specific branches: a routine scheduling path and one emergency-escalation variant. Critically, however, the demo path calls the *same* underlying tool functions, `_validate_address` and `_rank_providers`, that the production agent path calls (ADR-009). The only substitution is in the natural-language layer that decides what to say and when; the deterministic tool layer that actually validates data and enforces the emergency gate is shared, unmodified, between the two modes. As documented in ADR-009, demo mode will still perform a real Google Maps geocoding lookup unless the `GOOGLE_MAPS_API_KEY` environment variable is deliberately left unset, in which case address validation falls back to a fixed positive response; a fully offline demonstration therefore requires that explicit step.

The practical consequence is that a reviewer exercising the demo is, for every tool-mediated decision, observing the actual production code path rather than a stand-in for it. If the tools' behavior were to change, both modes would change together, since there is only one implementation to change — eliminating the possibility that the demo silently diverges from what is deployed.

Figure 2 shows a representative transcript of the terminal session produced by the demo mode described above, illustrating the tool-grounded conversation flow end to end.

VI. Evaluation

We report no formal clinical or user-study evaluation of this system, and we state this directly rather than implying rigor that does not exist. The project is, by its own documentation, a portfolio-scale engineering demonstration of an architectural pattern, evaluated to date only through basic software-quality gates: the source compiles cleanly (`py_compile`), the demo mode runs end-to-end without external API keys, the OpenAI tool schemas conform to the function-calling specification, the audit log and environment file are verified as excluded

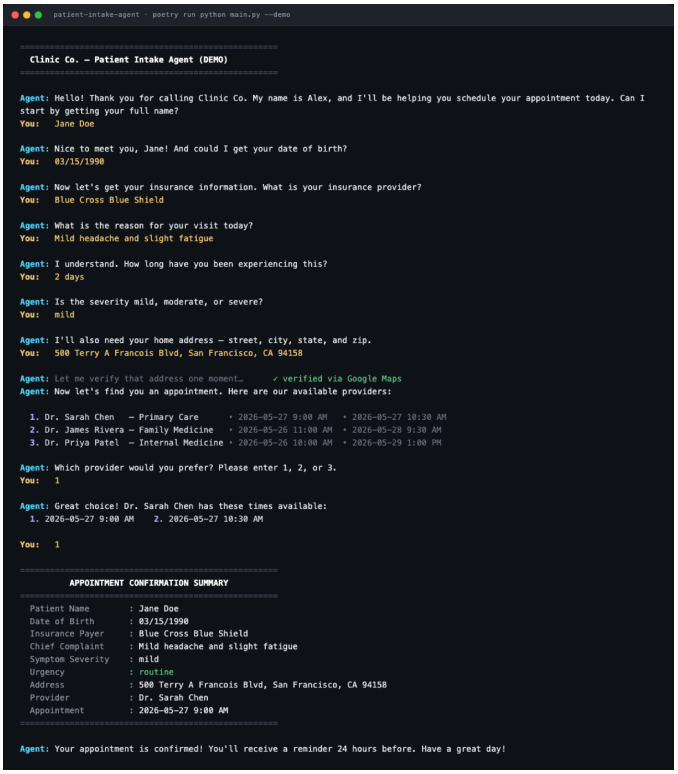

Fig. 2. A full transcript of the keyless demo session, showing the agent collecting patient information, invoking the address-validation and provider-ranking tools, and enforcing the two-layer emergency escalation gate described in Section III over the course of a single terminal-based conversation.

from version control, and the emergency-refusal property of `rank_providers` is checked directly by unit-level inspection of its return value under an emergency urgency argument. These are engineering correctness checks, not evidence of clinical safety or of triage accuracy against ground truth.

A rigorous evaluation of a system of this kind, prior to any real-world deployment, would need at minimum:

- **Clinical validation of the triage classification** by qualified medical personnel, comparing the LLM's emergency/urgent/routine classification against expert judgment across a representative and adversarial corpus of symptom descriptions, including ambiguous and edge-case phrasing.
- **A confusion-matrix-style analysis** of triage outcomes, with particular attention to false negatives on the emergency class, since a missed emergency is the system's highest-severity failure mode.
- **Red-teaming of the escalation gate itself** — adversarial or unusual phrasing intended to cause the model to misclassify an emergency, to verify that the tool-layer refusal in `rank_providers` does in fact hold even when the system-prompt layer fails.
- **User studies with real patients** measuring comprehension, trust, completion rate, and drop-off, none of which are measured by the current system (the project's own limitations documentation notes that time-to-complete, drop-off rate, and classification confusion are all currently unmeasured).
- **Operational reliability testing** under transient failure conditions (e.g., OpenAI API errors), since the current implementation has no retry logic and a transient 503 currently surfaces as a crashed session.
- **Regulatory and compliance review**, since a system of this kind, if it were to actually assist with clinical triage in production, would plausibly fall under medical device regulation (e.g., FDA Class II Software as a Medical Device or equivalent) and would require a HIPAA-compliant infrastructure review.

None of this evaluation has been performed, and the project's documentation is explicit that the gap between the current workflow demonstration and a real, clinically deployable system is significant.

VII. Limitations

The following limitations are drawn directly from the project's own documentation and are stated here without alteration of scope, because we consider honest limitation disclosure to be evidence of the project's engineering maturity rather than a weakness to obscure.

A. Scope limitations

The system is terminal-only, with no voice, SMS, or web interface. It operates over three mock providers with three mock slots each, and is not connected to a real scheduling system. Triage classification is keyword- and reasoning-driven within the LLM and is explicitly not clinically validated. The system is English-only, and no accessibility audit (e.g., screen-reader compatibility) has been performed.

B. Safety and clinical limitations

The agent is not a certified clinical triage tool; its 911 directive on emergency-consistent input is LLM-classified, not certified, and the system should be treated as a workflow demonstration rather than clinical decision support. The system is not HIPAA-compliant: the audit log is written to local disk in plaintext, with no encryption at rest, access controls, tamper-evidence, or business-associate agreements with any cloud provider. There is no PII redaction — patient-provided names, addresses, and symptoms are logged verbatim.

C. LLM-dependency limitations

Agent mode requires GPT-4o specifically, with no graceful degradation to a smaller or cheaper model. There is no retry logic on transient OpenAI errors, so a transient failure (e.g., HTTP 503) surfaces as a crashed session. There is no structured validation of the model's tool-call output; a malformed tool call would crash the dispatch function rather than degrade gracefully. Token cost is unbounded, as there is no maximum-turns counter on a conversation.

D. Operational limitations

The system is single-process with no concurrency (one terminal session at a time), retains no durable state across sessions, generates no staff alerting when a session ends in emergency status, and collects no metrics such as completion time, drop-off rate, or a triage confusion matrix.

E. Demo-mode limitations

The scripted demo covers exactly one routine-scheduling path and one emergency variant; it does not exercise all branches of the system. As noted in Section IV, the demo mode still performs a live Google Maps geocoding call unless the corresponding API key environment variable is explicitly unset.

F. What would be required to ship for real

The project's own documentation lists seven prerequisites for real-world deployment: clinical validation of the triage rules by qualified medical personnel; HIPAA-compliant infrastructure, including encrypted storage, business-associate agreements, and access controls; real EHR / scheduling-system integration in place of the mock provider table; multilingual support; voice and SMS intake surfaces, since most patients would not intake via a terminal; liability insurance; and a regulatory pathway such as FDA Class II Software as a Medical Device classification or its equivalent.

VIII. Conclusion and Future Work

We have presented a small, single-file, terminal-based patient intake agent that illustrates a specific and, we argue, broadly useful architectural pattern for safety-adjacent LLM applications: the language model is responsible for conversation, while every data-bearing operation — address validation and provider ranking — is delegated to deterministic, independently verifiable Python tools. The system's principal contribution is its two-layer emergency escalation gate, in which a system-prompt-level instruction to redirect emergency symptoms to 911 is reinforced by an independent, model-agnostic refusal inside the provider-ranking tool itself, so that the escalation property does not depend on the language model behaving correctly on every turn. A keyless demo mode calling the identical tool layer as production further ensures that what is demonstrated cannot diverge from what is deployed.

We have been explicit throughout that this is a portfolio-scale engineering demonstration, not a clinically validated or HIPAA-compliant system, and we regard the project's own thorough self-documentation of its limitations — spanning scope, safety, LLM dependency, operational maturity, and demo fidelity — as a positive signal about the rigor with which it was built, rather than as a deficiency to be minimized.

The project's own roadmap outlines a concrete path toward greater maturity: robustness improvements such as retry/backoff logic, a maximum-turns cost cap, and tool-call schema validation; additional intake surfaces (a FastAPI web interface, Twilio-based SMS, and Twilio/Whisper/ElevenLabs-based voice) built on the same tool layer; replacement of the mock provider table with a real clinic scheduling integration (e.g., an HL7 FHIR endpoint) without changing the tool schema exposed to the LLM; multilingual support; a confidence-scored escalation path that routes low-confidence triage cases to human nurse review rather than to automatic scheduling; and, longer term, a compliance harness covering tamper-evident encrypted audit logging, PII redaction in non-clinical logging contexts, and a business-associate-agreement-ready cloud deployment. Realizing this roadmap, together with the formal clinical evaluation outlined in Section V, represents the actual distance between the current demonstration and a system that could be responsibly deployed with real patients.